之前说过，我们在设计一个 xx-former 模型时，我们可以在 token mixer 部分，改进一个模块，这个算一个主要创新点，然后在 FFN 部分改进一个模块，这算一个次创新点。

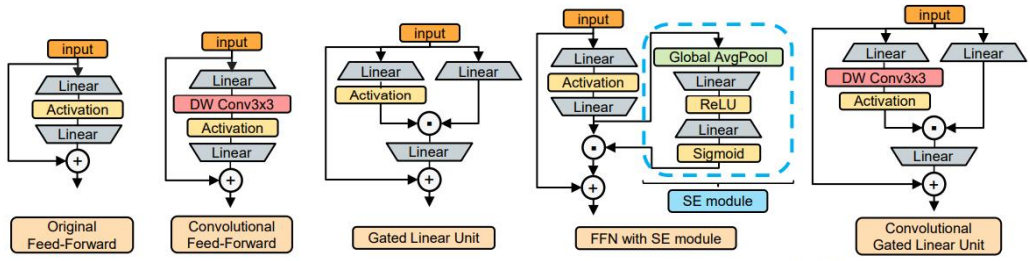

Figure 4. Comparison of prevalent channel mixer designs and Convolutional GLU

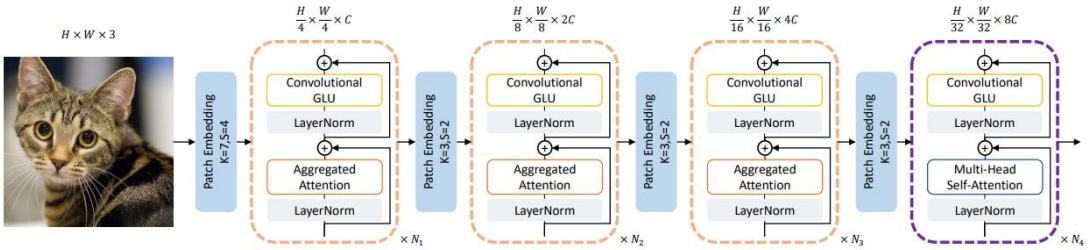

Figure 5. An illustration of TrasnNeXt architecture.

TransNeXt: Robust Foveal Visual Perception for Vision Transformers (CVPR2024)

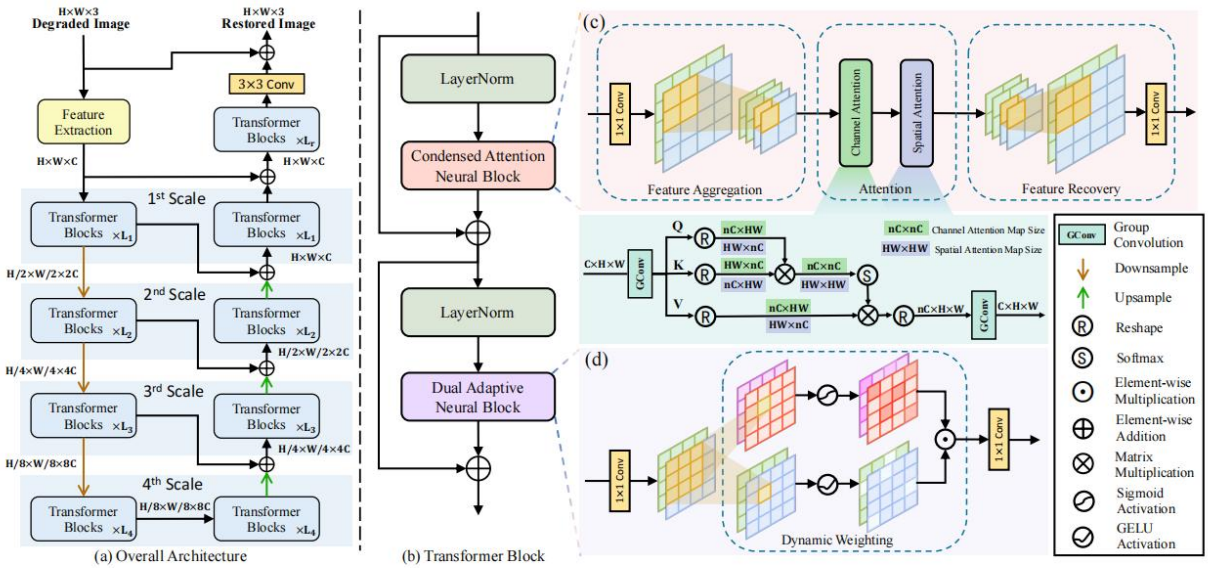

Figure 2. The overall architecture of our proposed efficient Transformer for image restoration. The key components are shown in (b) Transformer block which consists of (c) condensed attention neural block (CA) and (d) dual adaptive neural block (DA).

Comprehensive and Delicate: An Efficient Transformer for Image Restoration(CVPR2023)

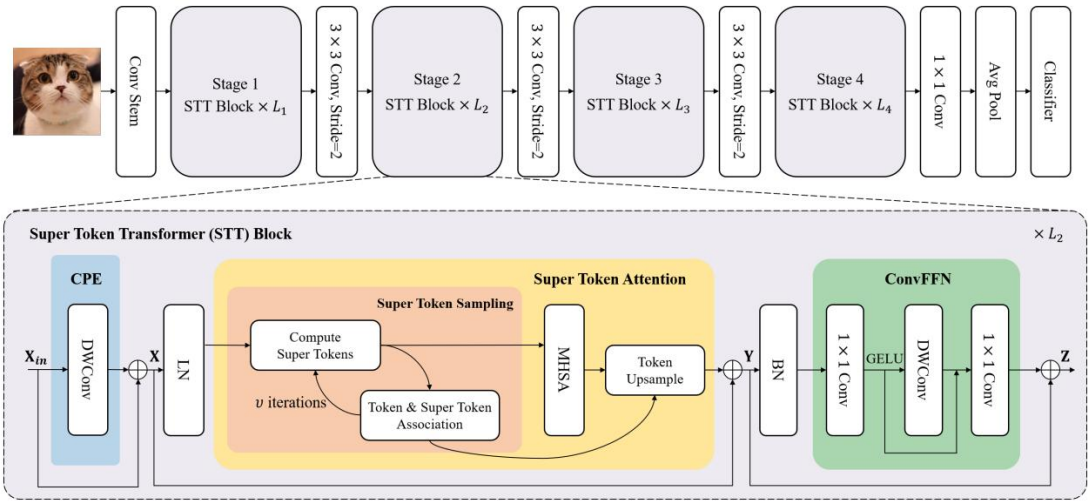

Figure 3. The architecture of Super Token Vision Transformer (SViT).

Vision Transformer with Super Token Sampling（CVPR203)

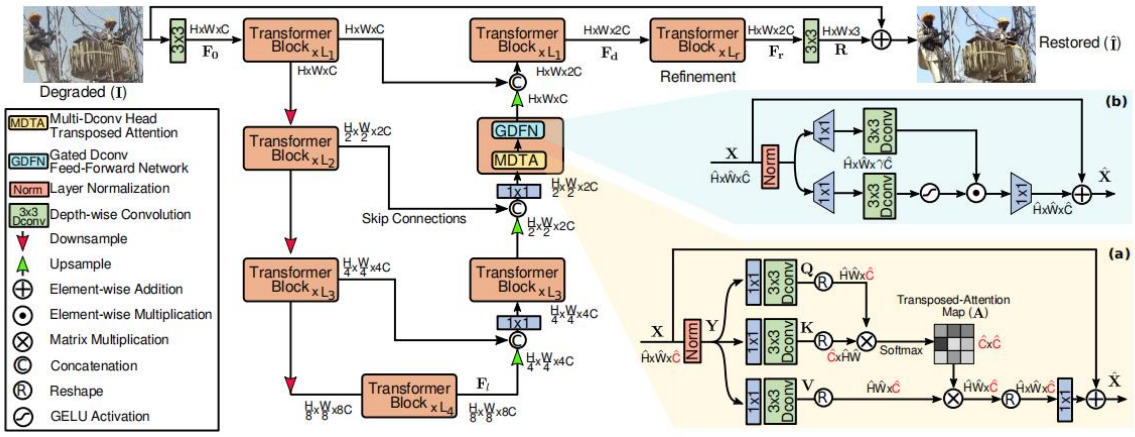

Figure 2. Architecture of Restormer for high-resolution image restoration. Our Restormer consists of multiscale hierarchical design incorporating efficient Transformer blocks. The core modules of Transformer block are: **(a)** multi-Dconv head transposed attention (MDTA) that performs (spatially enriched) *query-key* feature interaction across channels rather the spatial dimension, and **(b)** Gated-Dconv feed-forward network (GDFN) that performs controlled feature transformation, *i.e.*, to allow useful information to propagate further.

Restormer: Efficient Transformer for High-Resolution Image Restoration（CVPR2022)

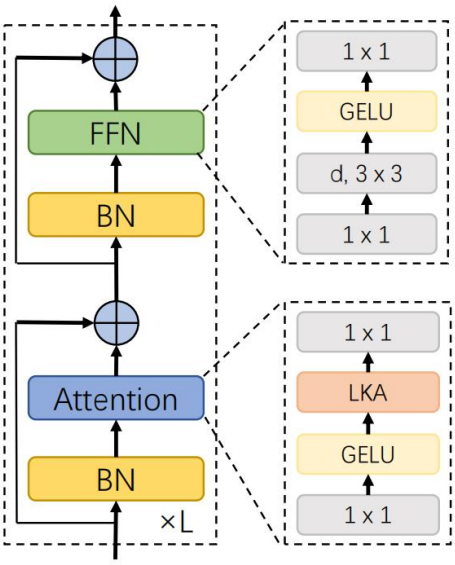

Fig. 4. A stage of VAN. d means depth wise convolution. k × k denotes k × k convolution.

Visual Attention Network (2022）

上述都是改进 FFN 层的一些例子，改进 FFN 层代码一般都比较简单，只有十几行，所以，我们改进起来也比较简单。例如加入卷积引入局部信息，或者使用门控机制等。我们可以简单的替换一下这些即插即用的 FFN 模块试试效果，在此基础上改进一下，变成自己提出的模块。

```python
# 论文：Comprehensive and Delicate: An Efficient Transformer for Image Restoration (CVPR2023)
# GitHub地址：https://github.com/XLearning-SCU/2023-CVPR-CODE/tree/main
# Dual adaptive neural block (DA).
class DualAdaptiveNeuralBlock(nn.Module):
    def __init__(self, embed_dim):
        super(DualAdaptiveNeuralBlock, self).__init__()
        self.embed_dim = embed_dim

        self.group_conv = nn.Sequential(
            nn.Conv2d(embed_dim, embed_dim, 1),
            nn.Conv2d(embed_dim, embed_dim * 2, 7, 1, 3, groups=embed_dim)
        )
        self.post_conv = nn.Conv2d(embed_dim, embed_dim, 1)

    def forward(self, x):
        B, C, H, W = x.size()
        x0, x1 = self.group_conv(x).view(B, C, 2, H, W).chunk(2, dim=2)
        x_ = F.gelu(x0.squeeze(2)) * torch.sigmoid(x1.squeeze(2))
        x_ = self.post_conv(x_)
        return x_
```